Molecular Featurization III: Descriptors

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang

From Lecture 25

Molecular Featurization II: Fingerprints

- *Morgan fingerprints and local atomic environments*
- *Generating fingerprints with RDKit*
- *Comparing molecules using fingerprint similarity*

In this lecture, you will learn

- *Generate and compare Morgan, MACCS, and RDKit fingerprints*
- *Calculate hydrogen-bond counts, TPSA, and calculated logP*
- *Construct a descriptor matrix*

Fingerprint Families

The feature definitions determine which structural differences affect the fingerprint.

Fingerprint	Structural features	Encoding
Morgan	Atom-centered neighborhoods	Hashed features
MACCS	Predefined structural conditions	A bit for each condition
RDKit	Bond paths and branched subgraphs	Hashed features

```
from rdkit.Chem import MACCSkeys, rdFingerprintGenerator
```

MACCS Keys

MACCS uses a fixed list of structural conditions.

- *Set a bit to 1 when its condition is satisfied.*
- *Leave it at 0 otherwise.*
- *The same bit tests the same condition in every molecule.*

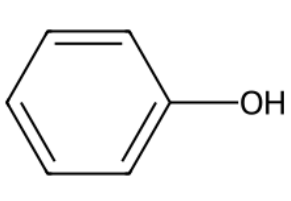
MACCS key	Chemical question
139	Is there an oxygen carrying at least one hydrogen?
159	Are there at least two oxygen atoms?
162	Is there an aromatic atom?
164	Is there an oxygen atom?

```
phenol = Chem.MolFromSmiles('Oc1ccccc1')
maccs_fp = MACCSkeys.GenMACCSKeys(phenol)

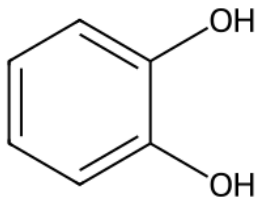
len(maccs_fp)
167
```

- The public MACCS set contains 166 keys.
- RDKit returns 167 positions because position 0 is unused (always 0).

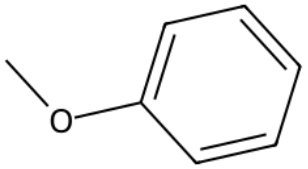
MACCS Keys



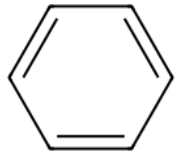
Phenol



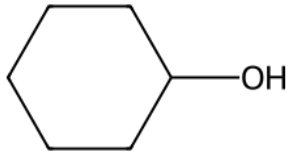
Catechol



Anisole



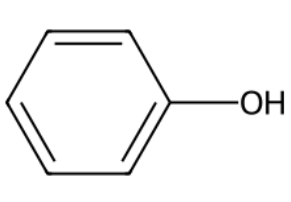
Benzene



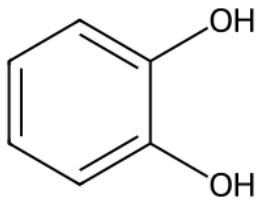
Cyclohexanol

Molecule	O–H present? 139	≥ 2 O atoms? 159	Aromatic atom? 162	O present? 164
Phenol				
Catechol				
Anisole				
Benzene				
Cyclohexanol				

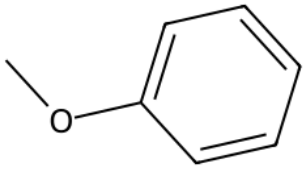
MACCS Keys



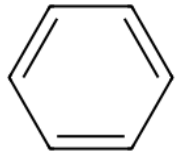
Phenol



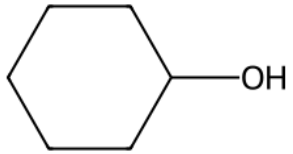
Catechol



Anisole



Benzene



Cyclohexanol

Molecule	O–H present? 139	≥ 2 O atoms? 159	Aromatic atom? 162	O present? 164
Phenol	1	0	1	1
Catechol	1	1	1	1
Anisole	0	0	1	1
Benzene	0	0	1	0
Cyclohexanol	1	0	0	1

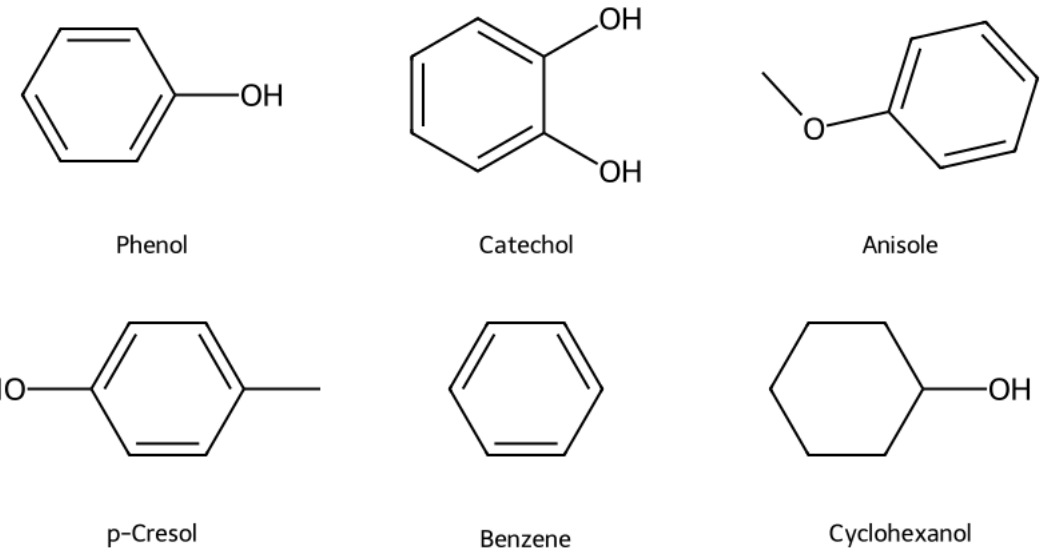
MACCS Keys

```
draw_options = Draw.MolDrawOptions()
draw_options.useBWAtomPalette()
draw_options.legendFontSize = 18
draw_options.legendFraction = 0.20

smiles = {
    'Phenol': 'Oc1ccccc1',
    'Catechol': 'Oc1ccccc1O',
    'Anisole': 'COc1ccccc1',
    'p-Cresol': 'Cc1ccc(O)cc1',
    'Benzene': 'c1ccccc1',
    'Cyclohexanol': 'OC1CCCCC1',
}

molecules = {name: Chem.MolFromSmiles(s) for name, s in smiles.items()}

Draw.MolsToGridImage(
    list(molecules.values()), legends=list(molecules),
    molsPerRow=3, subImgSize=(280, 220), drawOptions=draw_options,
)
```



MACCS Keys

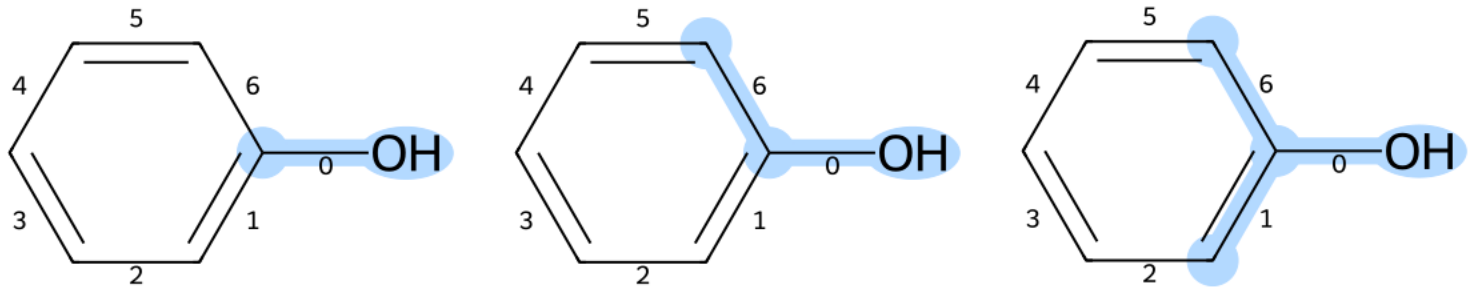
```
selected_keys = [139, 159, 162, 164]
key_rows = []
for mol in molecules.values():
    fingerprint = MACCSkeys.GenMACCSKeys(mol)
    key_rows.append([fingerprint[key] for key in selected_keys])

maccs_examples = pd.DataFrame(
    key_rows, index=list(molecules), columns=selected_keys,
)
maccs_examples
```

Molecule	O–H present? 139	≥ 2 O atoms? 159	Aromatic atom? 162	O present? 164
Phenol	1	0	1	1
Catechol	1	1	1	1
Anisole	0	0	1	1
Benzene	0	0	1	0
Cyclohexanol	1	0	0	1

RDKit Fingerprint

RDKit identifies small connected fragments in a molecule, including linear and branched structures. Each fragment is converted into positions in a binary fingerprint using a hash function.



Highlighted fragment	What it contains	Example bits activated
1 bond	Oxygen bonded to an aromatic carbon	1371, 1400
2 bonds	The O–C bond and one adjacent aromatic bond	959, 1342
3 bonds, branched	The O–C bond and both adjacent aromatic bonds	921, 954

By default, each fragment activates two bit positions.

RDKit Fingerprint

For each fragment, RDKit:

1. Describes its atom types, bond types, and connections.
2. Uses hashing to generate bit positions.
3. Sets those positions to 1.

```
from rdkit.Chem import rdFingerprintGenerator

path = rdFingerprintGenerator.GetRDKitFPGenerator(
    minPath=1, maxPath=7, fpSize=2048,
    branchedPaths=True, numBitsPerFeature=2,
)

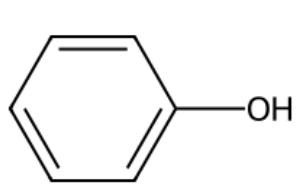
bit_info = rdFingerprintGenerator.AdditionalOutput()
bit_info.AllocateBitPaths()

rdkit_fp = path.GetFingerprint(phenol, additionalOutput=bit_info)
len(rdkit_fp)

2048
```

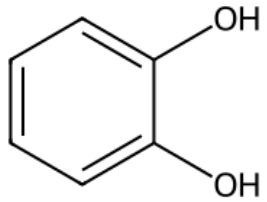
Setting	Meaning
minPath=1	Include fragments containing at least one bond
maxPath=7	Include fragments containing up to seven bonds
branchedPaths=True	Include branched fragments as well as linear paths
fpSize=2048	Store the fingerprint in 2048 bit positions
numBitsPerFeature=2	Generate two bit positions for each feature

Fingerprint Choice Changes the Similarity Ranking

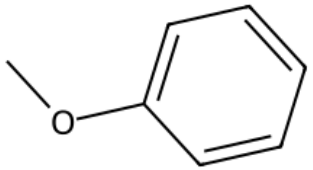


Phenol

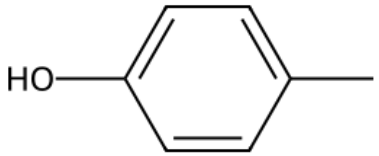
Reference



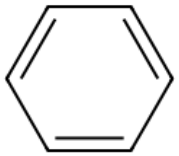
Catechol



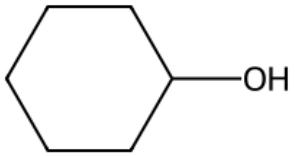
Anisole



p-Cresol



Benzene



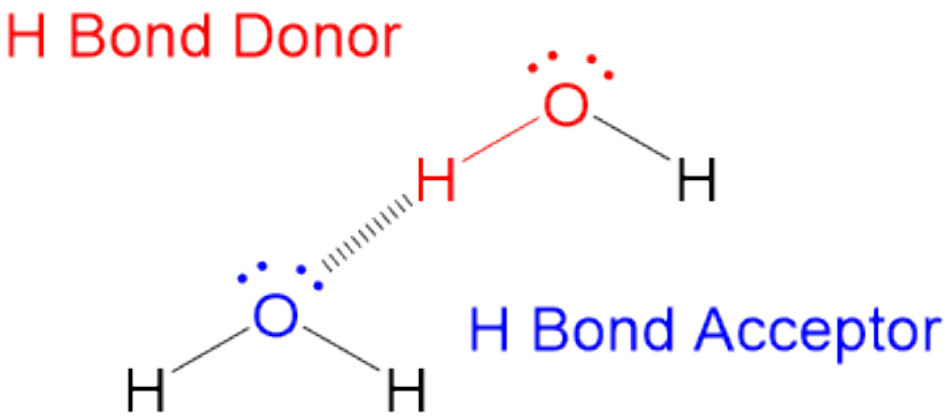
Cyclohexanol

Tanimoto Similarity $$T(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{x}_i^T \mathbf{x}_j}{\|\mathbf{x}_i\|^2 + \|\mathbf{x}_j\|^2 - \mathbf{x}_i^T \mathbf{x}_j}$$

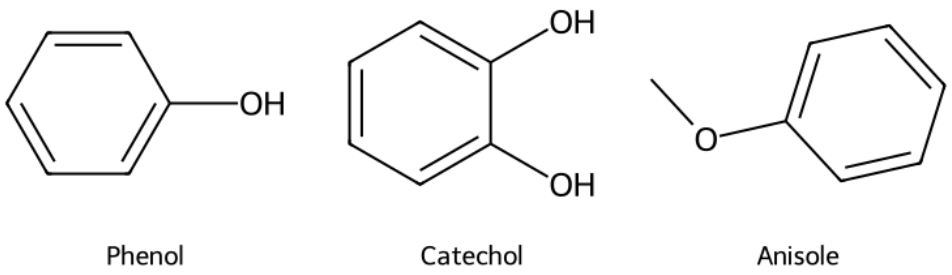
	Morgan	MACCS	RDKit
Catechol	**0.500**	0.625	**0.667**
Anisole	0.316	0.692	0.594
p-Cresol	0.412	**0.909**	0.507
Benzene	0.273	0.300	0.316
Cyclohexanol	0.048	0.444	0.000

Molecular similarity is representation-dependent.

Molecular Descriptors: Hydrogen-Bond Donors and Acceptors



- A *donor site* contains a hydrogen that can participate in hydrogen bonding.
- An *acceptor site* has a lone pair available for hydrogen bonding.



Phenol Catechol Anisole

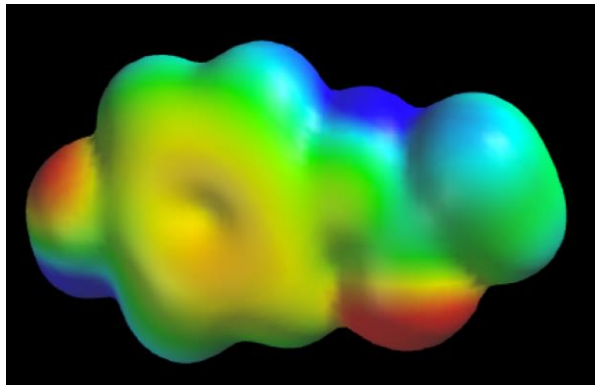
```
from rdkit.Chem import Lipinski

Lipinski.NumHDonors(phenol)
1

Lipinski.NumHAcceptors(phenol)
1
```

Molecule	Oxygen-containing groups	HBD	HBA
Phenol	One O–H group	1	1
Catechol	Two O–H groups	2	2
Anisole	One ether oxygen, no O–H	0	1

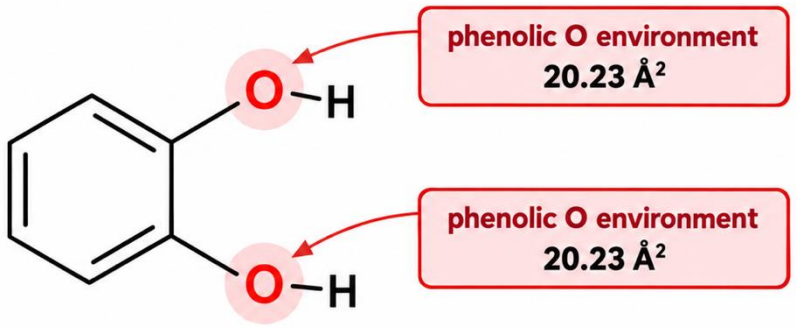
TPSA: Contributions from Polar Atom Environments



- *Topological Polar Surface Area (TPSA)* estimates polar surface area by adding contributions assigned to atom environments.

```
from rdkit.Chem import Descriptors

rdMolDescriptors.CalcTPSA(catechol)
40.46
```



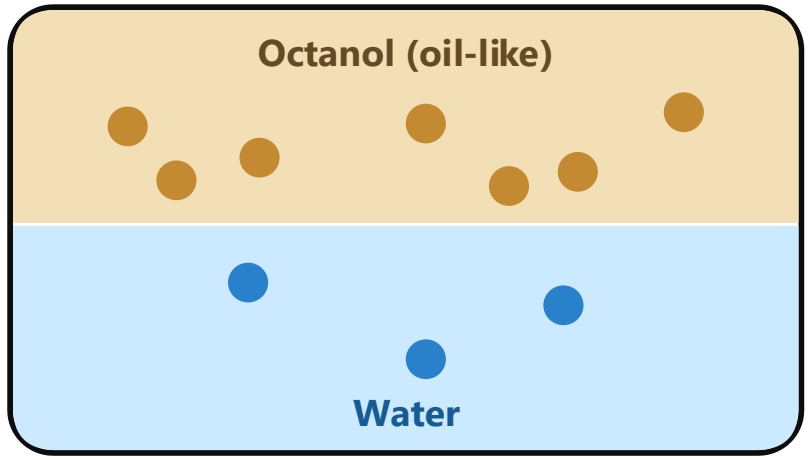
Molecule	Relevant oxygen environments	TPSA calculation
Phenol	One hydroxyl oxygen	1 * 20.23 = 20.23 Å²
Catechol	Two hydroxyl oxygens	2 * 20.23 = 40.46 Å²
Anisole	One ether oxygen	1 * 9.23 = 9.23 Å²

- The oxygen's bonding environment affects its contribution.
- The calculation uses connectivity and atom environments. It requires no 3D coordinates.

logP

LogP measures how strongly a molecule prefers a nonpolar environment (oil) over water.

$$P = \frac{c_{\text{octanol}}}{c_{\text{water}}}, \qquad \log P = \log_{10} P$$



logP	P	Concentration ratio
0	1	Equal concentrations
1	10	Ten times higher in octanol
2	100	One hundred times higher in octanol

It is a prediction, not an experimental measurement.

```
from rdkit.Chem import Crippen

round(Crippen.MolLogP(phenol), 4)
1.3922
```

Interactive Python Time

26_molecular_descriptors.ipynb

In this lecture, we learned

- *Generate and compare Morgan, MACCS, and RDKit fingerprints*
- *Calculate hydrogen-bond counts, TPSA, and calculated logP*
- *Construct a descriptor matrix*

Next lecture:

- *Molecular property prediction*